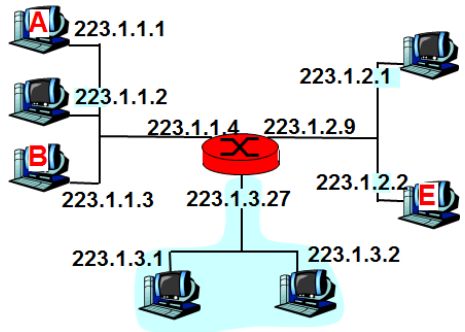
Redes de Datos II
QUIC

Matías Robles

LINTI–UNLP

22 de septiembre de 2026



CC BY NC SA

Contenidos

1. QUIC - Introducción
2. Paquetes, Frames y Streams
3. Conexiones
4. Control de Flujo
5. Seguridad

Estamos en:

1 QUIC - Introducción

2 Paquetes, Frames y Streams

3 Conexiones

4 Control de Flujo

5 Seguridad

Problemas en TCP

- TCP ha presentado varios problemas a través de los años:
  - **Head of Line blocking(HLB):** pérdida de un segmento bloquea el envío de los siguientes segmentos hasta que sea recuperado.
  - **Retardo debido al establecimiento de la conexión:** saludo de 3-vías antes de enviar datos. Si se usa TLS, se requiere otro RTT (Round Trip-Time).
  - **Limitaciones debido al tamaño de cabecera fijo:** TCP tiene un tamaño de cabecera de 20 bytes fijos más 40 bytes de opciones.
  - **Identificación único de conexiones:** dirección IP y puerto TCP para identificar una conexión. Se pierde si cambia la IP.

QUIC - Historia

- Desarrollado inicialmente por Jim Roskind en Google en 2012.
- 2013, Google lo hizo público y se lo envió a la IETF.
- 2015, la IETF establece QUIC Working Group.
- 2014, Google considera desplegarlo gradualmente.
- 2017, QUIC es usado en prácticamente todos los navegadores Chrome
- 2021, la IETF anuncia la RFC 9000 - QUIC: A UDP-Based Multiplexed and Secure Transport, que es la propuesta de estándar del protocolo.

QUIC - Introducción

- Protocolo de transporte seguro orientado a conexión.
- Extremos establecen una conexión con estado.
- Establecimiento de la sesión combina negociación de parámetros de transporte y criptografía.
- Diseñado para permitir el intercambio de datos lo más rápido posible (0-RTT).
- Intercambia paquetes QUIC que viajan en datagramas UDP.
- En el SO, se implementa en el espacio del usuario.
- Paquetes QUIC contienen frames que transportan información de control y datos de la aplicación.
- Autentica la totalidad de cada paquete y encripta tanto como sea posible.
- Protocolos de aplicación intercambian información en una conexión QUIC usando *streams*.

Estamos en:

1 QUIC - Introducción

2 Paquetes, Frames y Streams

3 Conexiones

4 Control de Flujo

5 Seguridad

QUIC - Streams

- Son una abstracción liviana dentro de una conexión QUIC que representan un flujo de bytes confiable y ordenada.
- Pueden ser unidireccionales o bidireccionales.
- Se pueden crear mediante el envío de datos.
- Mínimo overhead en los procedimientos de administración (cancelar, finalizar, etc.).
- Un stream puede durar todo lo que perdure la conexión.
- Pueden ser creados por cualquiera de los extremos.
- QUIC permite que exista un número arbitrario de streams concurrentemente.
- Datos de diferentes streams se pueden enviar intercalados.

QUIC - Streams . . .

- Identificados con un stream ID único de 62 bits (0 a 2^{62} - 1).
- Un stream ID no debe ser reusado por un extremo.
- Último bit del stream ID identifica el iniciador del stream. Cliente usan IDs pares, servidores impares.
- Anteúltimo bit indica si el stream es bidireccional (0) o unidireccional (1).

Bits	Iniciado por	Tipo
0x00	Cliente	Bidireccional
0x01	Servidor	Bidireccional
0x02	Cliente	Unidireccional
0x03	Servidor	Unidireccional

QUIC - Streams . . .

- Frames *STREAM* encapsulan datos enviados por las aplicaciones.
- Extremos usan el stream ID y los campos Offset y Length para ubicar los datos en orden.
- Datos se deben pasar ordenados a la aplicación.
- Uso de buffers para almacenar datos fuera de orden.
- Límites de un frame *STREAM* puede no ser preservados cuando los datos se envían, retransmiten o se pasan a la aplicación.
- Datos enviados en un stream deben estar dentro de los límite de control de flujo indicados por el otro extremo.
- Cada extremo controla la cantidad de streams concurrentes que su peer puede utilizar.
- Aplicación podría indicar priorización de los streams. QUIC determina como asignar recursos.

Streams States

- La parte emisora de un stream que inicia un extremo es abierta por la aplicación, estado *Ready*.
- Al enviar el primer frame *STREAM*, el extremo pasa del estado *Ready* a *Send*.
- En ese estado, el extremo transmite datos de un stream en frames *STREAM*.
- Un extremo envía un frame *STREAM* con el bit FIN seteado cuando la aplicación le indica que todos los datos han sido enviados. Pasa al estado *Data Sent*.
- Al recibir un ACK para todos los datos enviados, el extremo pasa al estado *Data Recvd*.
- Desde los estados *Ready*, *Send* o *Data Sent* un extremo puede enviar un frame *RESET_STREAM*.
- Al ser reconocido el *RESET_STREAM*, el extremo pasa al estado *Reset Recvd*, que es un estado terminal.

Streams States . . .

- La parte receptora de un stream iniciado por el otro extremo es creada al recibir el primer frame *STREAM* o *RESET_STREAM*.
- En el estado *Recv*, el extremo pone en un buffer los datos recibidos y se los pasa a la aplicación.
- A medida que los datos son consumidos, el extremo envía frames *MAX_STREAM_DATA* para habilitar al otro extremos a enviar más datos.
- Al recibir un frame *STREAM* con el bit FIN seteado, el extremo conoce el tamaño total del stream y pasa al estado *Size Know*.
- Al recibir todos los datos del stream, se mueve al estado *Data Recvd*.
- Una vez que todos los datos se pasaron a la aplicación, el stream entra en el estado *Data Read*.
- Al recibir un frame *RESET_STREAM*, el stream se mueve al estado Reset Recvd".

Paquetes y Frames

- Extremos se comunican mediante el intercambio de paquetes QUIC que contienen frames.
- **Frame:** unidad estructurada de comunicación para transportar un tipo específico de datos. Diferentes tipos de frames.
- Paquetes pueden tener una cabecera corta o larga.
- Se usan paquetes con cabeceras largas durante el establecimiento de la conexión.
- Después del establecimiento de la conexión se pasa a utilizar la cabecera corta.
- Un datagrama UDP puede encapsular 1 o más paquetes QUIC.
- Un paquete QUIC puede encapsular 1 o más frames del mismo o diferentes streams.
- El envío confiable de datos es manejado a nivel de paquetes mas que a nivel de frame.

Paquetes

- Paquetes con cabeceras largas: *Initial*, *0-RTT*, *Handshake*, , *Version Negotiation* y *Retry*.
- Paquetes con cabeceras cortas: *1-RTT*.
- Por ej., paquetes con cabecera corta no contienen campo longitud.
- Paquetes tienen diferentes niveles de protección criptográficos basados en el tipo de paquete.
- Paquetes se numeran entre 0 y $2^{62} - 1$. Siempre inician en 0.
- Números de paquetes se dividen en 3 espacios:
  - **Initial Space:** comprende todos los paquetes iniciales.
  - **Handshake Space:** comprende todos los paquetes del handshake.
  - **Application Data Space:** comprende los paquetes 0-RTT y 1-RTT.
- Cada tipo de paquete usa diferentes claves de protección.

Paquetes - Tipos

- **Version Negotiation:** enviados solo por un servidor cuando no conoce la versión de QUIC indicada por el cliente.
- **Initial:** transporta el primer frame *CRYPTO* enviado por el cliente y el servidor para ejecutar el intercambio de claves.
- **0-RTT:** usado para enviar datos de la aplicación inmediatamente, antes que se complete el handshake. Requiere de una conexión previa.
- **Handshake:** usado para transportar mensajes criptográficos y ACKs del proceso de handshake.
- **Retry:** enviado por el servidor cuando desea validar la dirección IP del cliente (antes del proceso de intercambio de claves).
- **1-RTT:** usados después que la versión y las claves 1-RTT son negociadas. Contienen frames *STREAMS* y *ACK*. Payload siempre va protegido.

Frames

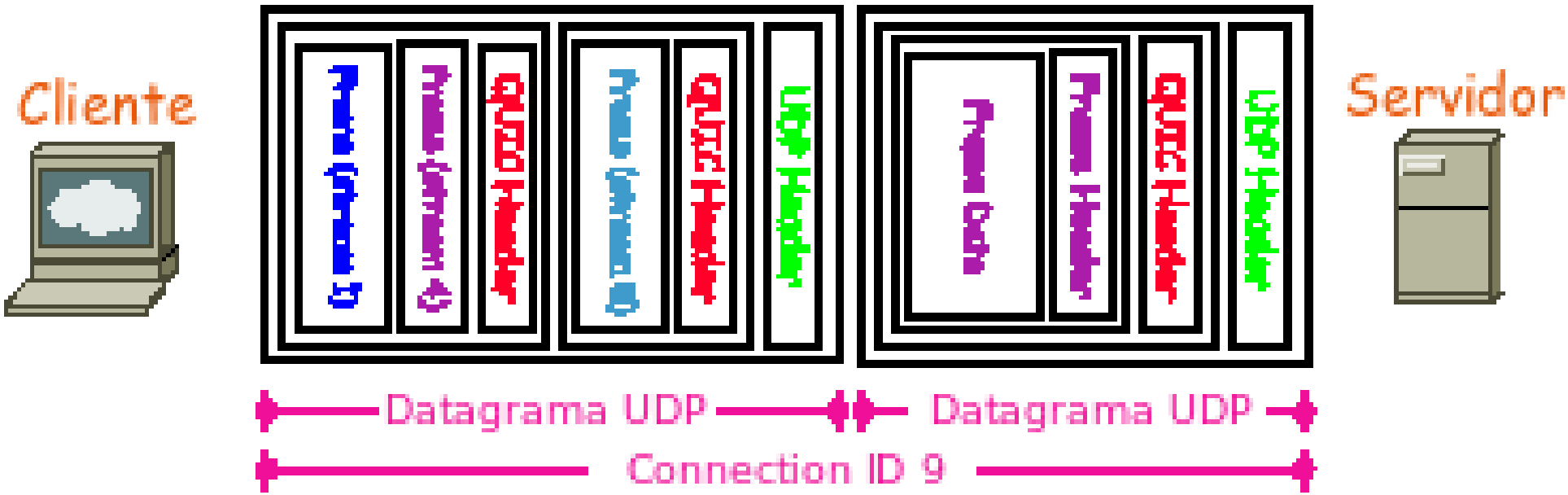
- El payload de la mayoría de los paquetes QUIC consiste de uno o varios frames.
- Algunos paquetes (Retry, Version Negotiation) no contienen frames.
- Un frame debe caber completamente en un paquete.
- Cada frame comienza con un campo *Frame Type* que indica su tipo.
- Algunos frames están prohibidos en determinados espacios de números de paquetes.
- Un único paquete QUIC puede incluir frames *STREAM* de uno o varios streams.
- Pérdida de un paquete bloquea todos los streams para los cuales contiene frames.

Tipos de Frames (algunos)

- **CRYPTO:** transmite mensajes criptográfico en el proceso de handshake.
- **STREAMS:** crea implícitamente un stream y transmite datos del stream.
- **ACK:** usado para reconocer paquetes (no frames).
- **RESET_STREAM:** termina abruptamente la parte emisora del stream.
- **PING:** usado para verificar que el otro extremo está vivo.
- **MAX_DATA:** usado en control de flujo para informar la cantidad máxima de datos que pueden ser enviadas en una conexión.
- **CONNECTION_CLOSE:** enviado para notificar al otro extremo que la conexión está siendo cerrada.

Paquete, Streams, Frames

- Multiplexación de streams es lograda intercalando frames *STREAMS* de múltiples streams dentro de uno o más paquetes QUIC.



Estamos en:

1 QUIC - Introducción

2 Paquetes, Frames y Streams

3 Conexiones

4 Control de Flujo

5 Seguridad

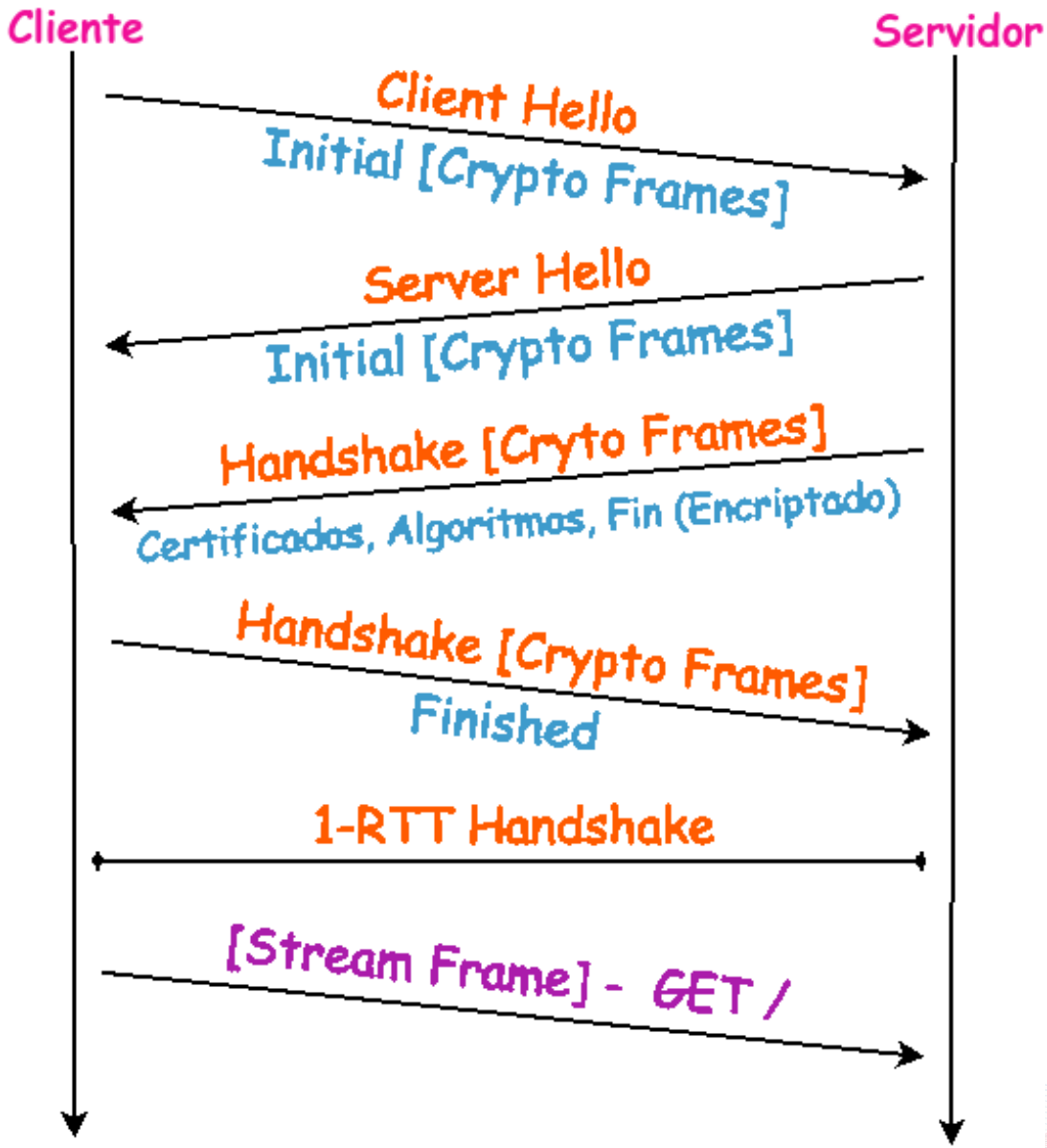
Connections ID (CID)

- Cada conexión posee un conjunto de identificadores de conexión, Connections ID (CID).
- Son elegidos en cada extremo de manera independiente.
- Cada extremo selecciona un CID que será usado por el otro extremo.
- Su principal función es asegurar que los paquetes de una conexión QUIC no sean enviados a un extremos erróneo si cambian las direcciones en los capas inferiores (UDP, IP).
- Paquetes con cabecera larga incluyen *Destination Connection ID* y *Source Connection ID*.
- Paquetes con cabecera corta solo incluyen *Source Connection ID*.

Establecimiento de conexión

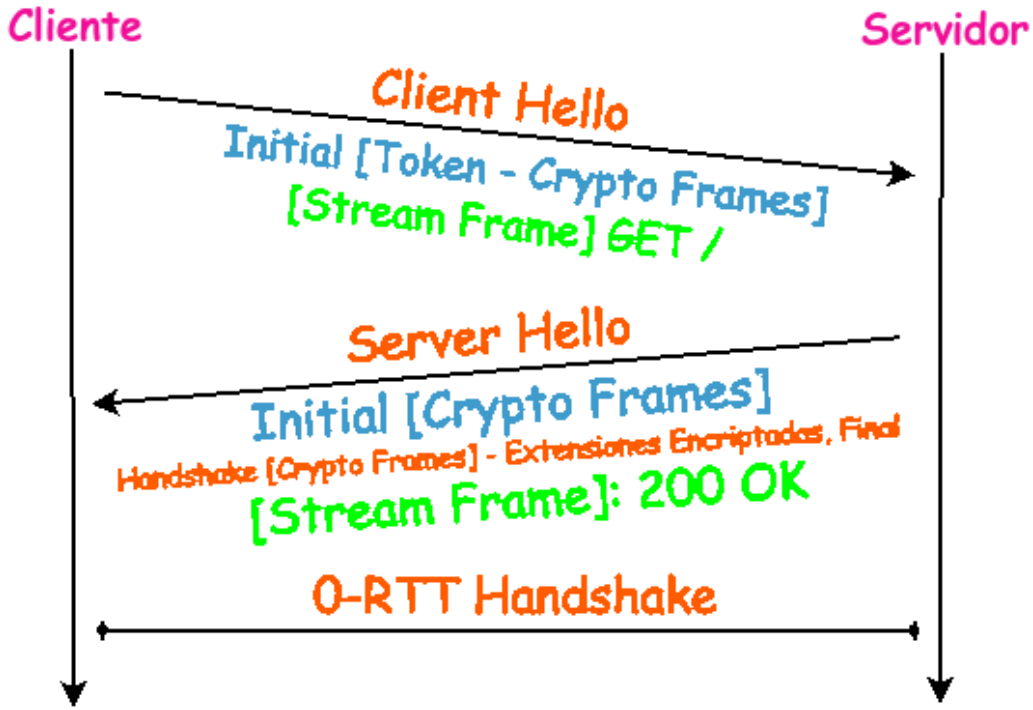
- QUIC combina los *handshakes* de transporte y criptográficos en un solo RTT (Round-Trip Time).
- Cada conexión comienza con una fase de saludo *handshake* en la cual se establece un secreto compartido entre los extremos.
- Conexiones son aseguradas usando TLS 1.3 (Transport Layer Security).
- QUIC utiliza frames *CRYPTO* para intercambiar información durante el *cryptographic handshake*.
- El handshake comienza con el cliente enviando un *Client Hello* de TLS 1.3 en un frame *CRYPTO* en un paquete QUIC *Initial*.
- El servidor responde con un *Server Hello* y toda la información de seguridad usando varios paqeuetes QUIC (Initial, Handshake y opcionalmente puede enviar datos en paquetes 1-RTT).
- Por último, el cliente finaliza el handshake.
- *TLS handshake* está completo si cada extremo ha enviado un mensaje *Finished* y ha verificado el mensaje *Finished* recibido.

1-RTT Handshake



0-RTT Handshake

- Permite a un cliente enviar datos de aplicación antes que el *handshake* esté completo.
- Reutiliza parámetros negociados en una conexión previa que el cliente recuerda.
- Provee un ticket de la sesión TLS al servidor para que pueda recuperar los mismos datos.



ACKs

- Reconocimiento de paquetes se envían en frames *ACK*.
- A diferencia de TCP, QUIC separa el orden de transmisión del orden de envío.
- Cada paquete está identificado por un número que debe ser reconocido por el otro extremo.
- Ese reconocimiento incluye todos los frames contenidos en ese paquete.
- Cada paquete QUIC en un datagrama UDP debe ser reconocido individualmente.
- Si se pierde un paquete QUIC todos los streams con datos en ese paquete se bloquean esperando por una retransmisión.
- Un paquete perdido nunca es retransmitido. Se genera otro paquete con un nuevo número de paquete.

ACKs . . .

- Frame ACK reconoce el último paquete recibido e indica los *gaps* que existen en el stream.
- Un extremo debe reconocer inmediatamente los paquetes *Initial* y *Handshake* (si ya dedujo las correspondientes claves).
- Paquetes *0-RTT* y *1-RTT* deben ser reconocidos dentro del tiempo *max_ack_delay* recibido.
- Un paquete no debe ser reconocido hasta que la protección del paquete ha sido removida exitosamente y todos los frames contenidos en el paquete han sido procesados.
- Al igual que en TCP, un ACK de un frame *STREAMS* no significa que los datos fueron recibidos por la aplicación.
- Una vez que el paquete ha sido completamente procesado, el receptor envía 1 o más frames *ACK* conteniendo el número del paquete del paquete recibido.

Cierre de conexiones

- Pueden ser finalizadas de 3 maneras diferentes:
  - Tiempo de inactividad
    - Cliente y servidor anuncian su tiempo de inactividad. Eligen el menor.
    - Conexión se cierra silenciosamente.
    - Un extremo reinicia el tiempo de inactividad cuando recibe y procesa exitosamente un paquete
  - Cierre inmediato
    - Extremo envía un frame *CONNECTION_CLOSE* para terminar una conexión inmediatamente.
    - Todos los streams se cierran en el acto.
    - Puede ser por una violación al protocolo o porque la aplicación lo solicita.
  - Reset sin estado
    - Opción de último recurso cuando un extremo no tiene acceso al estado de una conexión.
    - Extremo recibe paquetes que no puede asociar a una conexión activa.
- QUIC no provee un mecanismo para terminar un conexión de manera elegante.

Estamos en:

1. QUIC - Introducción
2. Paquetes, Frames y Streams
3. Conexiones
4. Control de Flujo
5. Seguridad

Control de Flujo

- Receptor necesita limitar la cantidad de datos que puede recibir y poner en su buffer.
- Dos niveles de control de flujo:
  - *Nivel de Stream:* previene que un stream consuma el buffer completo de la conexión.
  - *Nivel de Conexión:* previene que un emisor exceda la capacidad del buffer del receptor pero tiene en cuenta todos los streams.
- Emisor no debe exceder ninguno de los dos límites.
- Límites iniciales para todos los streams se setean durante el *handshake*.
- Un receptor envía frames *MAX_DATA* o *MAX_STREAM_DATA* al emisor para anunciar límites más grandes.

Control de Flujo . . .

- Si un receptor anuncia un límite menor al previamente anunciado no es un error, pero no tiene efecto.
- Un receptor debe cerrar una conexión con un error del tipo *FLOW_CONTROL_ERROR* si el emisor viola los límites informados.
- Un emisor no podrá enviar nuevos datos y se lo considera bloqueado si ya ha enviado datos hasta el límite.
- El emisor debería enviar un frame *STREAM_DATA_BLOCKED* o *DATA_BLOCKED* para indicarle al receptor que tiene datos para enviar pero está bloqueado por los límites de contro de flujo.
- El emisor debería enviar esos mensajes periódicamente para evitar el cierre de la conexión.

Control de Flujo . . .

- Para una conexión, el receptor indica el máximo absoluto de la suma de los deplazamientos (offset) de todos los streams.
- Receptor mantiene una suma acumulada de los bytes recibidos en todos los streams .
- Una conexión debe ser cerrada por un receptor si el emisor viola los límites anunciados tanto para la conexión como para uno de sus streams.
- Emisor debe ignorar cualquier frame que no incremente los límites del control de flujo.
- Si el emisor envió datos hasta el límite, no podrá enviar nuevos datos y se considera bloqueado.
- Debe avisar al receptor que tiene datos para enviar pero está bloqueado por los límites del control de flujo.

Estamos en:

1. QUIC - Introducción
2. Paquetes, Frames y Streams
3. Conexiones
4. Control de Flujo
5. Seguridad

Transport Layer Security (TLS) 1.3

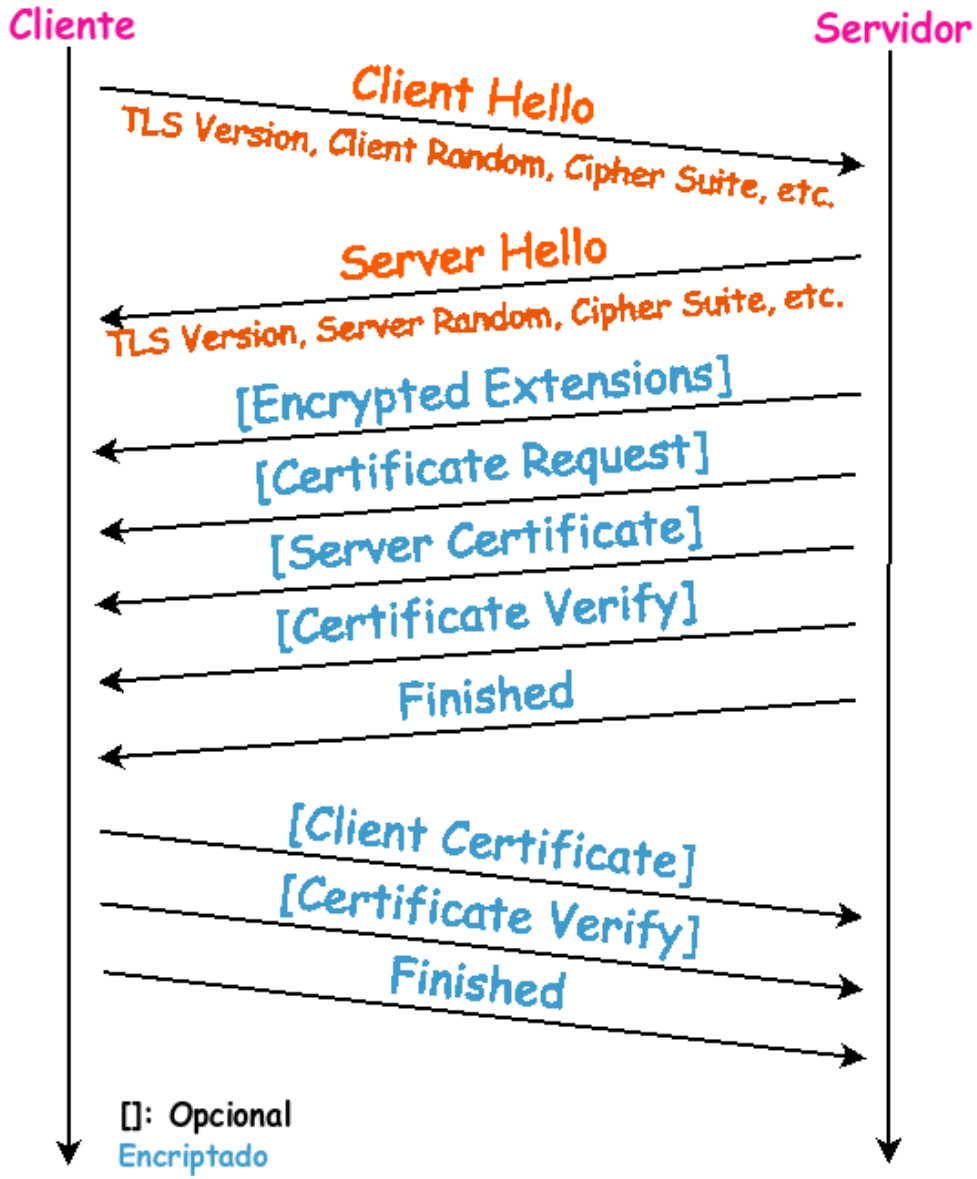
- Protocolo usado para proveer un canal seguro entre dos extremos.
- Provee autenticación del servidor al cliente. Opcionalmente, del cliente al servidor.
- Además brinda confidencialidad e integridad de los datos transmitido.
- Requiere un protocolo de transporte confiable para funcionar.
- QUIC se basa en TLS 1.3. No soporta otra versión.
- TLS 1.3 consiste principalmente de dos componentes:
  - **Handshake Protocol:** autenticación, negociación de protocolos criptográfico e intercambio de claves.
  - **Record Protocol:** permite la transferencia segura de datos usando las claves establecidas durante el handshake.

Transport Layer Security (TLS) 1.3 . . .

- Cada mensaje de la capa de *content layer* es transportada como un serie de *TLS records*.
- Cada registro es protegido individualmente y transmitido sobre un transporte confiable.
- Cliente inicia el intercambio autenticado de claves (PSK y Diffie-Hellman). Servidor responde.
- Si el intercambio es exitoso, cliente y servidor acuerdan un secreto.
- Una vez finalizado el TLS handshake, cliente habrá aprendido y autenticado al servidor (inverso opcional)
- TLS soporta autenticación basada en certificados X.509.

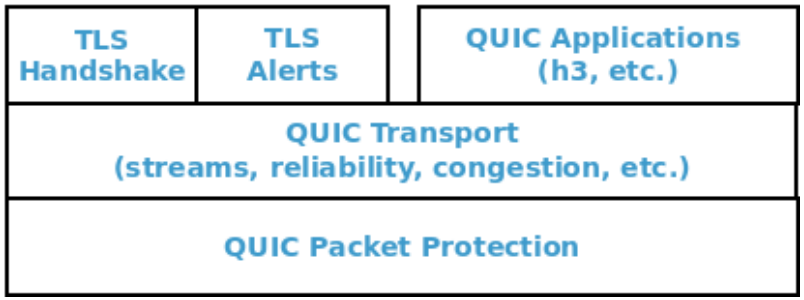


Transport Layer Security (TLS) 1.3 . . .



QUIC - TLS 1.3

- QUIC asume la responsabilidad de la confidencialidad e integridad de los paquetes utilizando las claves derivadas del TLS handshake.
- Mensajes *TLS handshake* y *TLS Alert* se envían sobre QUIC.
- QUIC también usa a TLS para la autenticación y negociación de parámetros necesarios para la seguridad y performance.
- No sigue un modelo de capas estricto, sino que cooperan entre sí: QUIC usa el TLS handshake, TLS el envío ordenado y la confiabilidad de QUIC.
- TLS actualiza QUIC con claves de protección, cambios de estados como finalización del handshake, certificado del servidor, etc.



QUIC - TLS 1.3

- QUIC transporta mensajes TLS en frames *CRYPTO*.
- Frames identificados por un offset y una longitud.
- Se empaqueta en paquetes QUIC y se encriptan con el nivel actual de encriptación.
- Cada pedazo de datos producidos por TLS se asocia al conjunto de claves que TLS está utilizando.
- Cada nivel de encriptación se corresponde a un espacio de número de paquetes.
- En cada espacio son permitidos determinados tipos de paquetes.
- Tipo de paquete indica que claves fueron usadas para protegerlo.

QUIC - TLS 1.3 . . .

Packet Type	Encryption Keys	PN Space
Initial	Initial Secrets	Initial
0-RTT Protected	0-RTT	Application Data
Handshake	Handshake	Handshake
Retry	Retry	NA
Version Negotation	NA	NA
Short Header	1-RTT	Application Data

- Paquetes *Initial* no se han negociados claves, pero se encriptan con una clave derivada del *Destination Connection ID*.
- 0-RTT reusa claves y parámetros de una conexión previa.
- Una vez intercambios los mensajes *ClientHello* y *ServerHello* ya se puede derivar las claves para el resto del handshake.
- Al completar el handshake se derivan las claves que se usarán en toda futura comunicación.

QUIC - TLS 1.3 . . .

- TLS handshke es manejado por TLS y usa QUIC como ”transporte”.
- Proceso de encriptación es realizado por QUIC usando las claves recibidas de TLS.
- Paquete consiste de una cabecera y payload (datos).
- El payload es encriptado directamente.
- **Header Protection:** determinadas partes de la cabecera son protegidas (por ej. número de paquete).
- Cabecera protegida y payload encriptado forman el paquete QUIC que ya puede ser ubicado en un datagrama UDP.



Esta obra está sujeta a la licencia Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0) de Creative Commons.

Para detalle de esta licencia visite https://creativecommons.org/licenses/by-nc-sa/4.0/